

Guide to CellDy: A Python Library to study the dynamics of Cellular Automata systems

Ignacio M. Pineda

July 2026

1 Introduction

This guide provides documentation for CellDy, a Python library developed for the study, simulation, and classification of classical, semiclassical, and quantum cellular automata systems (or CCA, SCA and QCA, respectively). The library offers a simplified and unified framework for constructing, evolving, analysing, and comparing cellular automata across different physical regimes.

CellDy is designed to facilitate the investigation of complex dynamical behaviour, including propagation, interactions, emergent phenomena, and the evolution of information within discrete systems. By providing a common computational framework for classical, semiclassical, and quantum cellular automata, the library aims to support both exploratory research and systematic analysis of their underlying dynamics.

The development of the library is conducted across 8 python notebooks and can be learnt by strictly using the notebooks. However, it is considered that a more concise guide is more useful for learning the framework and allows for additional comments on the development of CellDy.

In order to make the guide completely self-contained, all relevant topics involving historical context and concepts from physics and mathematics are presented in the section "Relevant Background". Nevertheless, it is still recommended to look at the original notebooks where the functions were located to look at the inner workings of the functions, examples and even things this guide may have skipped over. The associated GitHub repository is the following:

GitHub repository

2 List of Functions and Where to Find Them

Do when all else is done.

3 Relevant Background

It is also relevant to provide the necessary background, including a description and history of classical and quantum Cellular Automata, examples of important rule numbers and definitions of relevant concepts mostly related to general computer science, complexity theory and dynamical systems.

3.1 Classical and Quantum Cellular Automata

Cellular Automata were first conceived by physicist John Von Neumann and mathematician Stanislaw Ulam during the development of the hydrogen bomb in Los Alamos. Trying to find a solution to the question posed, regarding if a machine could build a copy of itself, Von Neumann, following Ulam's suggestions, devised a grid-based, discrete, mathematical space where each cell's behaviour would be governed by a local rule and all of the cell's evolution through time would give place to emergent phenomena.

Von Neumann's work gave place to a lot of further work, like Conway's game of life. However, the topic of discussion is the use of these classical Cellular Automata systems within the field of physics.

Towards the eighties, physicists started seeing the potential in CA, particularly as a way to simulate physical systems. In 1967, computer scientist and civil engineer Konrad Zuse proposes that the universe itself may be a cellular automaton. The first application of CA in physics is in the HPP model, a fully discrete boolean model of two-dimensional classical gas used to study the dynamics of said gas. However, it is in the eighties that it is demonstrated that CA can simulate physical models, including the HPP model. It is also around this timeframe that Stephen Wolfram establishes CA as a serious modeling tool for physical complexity with his publications "Cellular Automata as Models of Complexity" and "Universality and Complexity in Cellular Automata".

Additionally, in 1983 Wolfram publishes "Statistical Mechanics of Cellular Automata", where principles of statistical physics are used to study these discrete systems. It is also where the formal framework of CA is introduced and which acts as a foundation of CellDy. This includes the rule numbering scheme, the distinction between reversible and irreversible CA and the concept of the garden of Eden, randomness within the systems, and the characteristics of elementary cellular automata.

Subsequent work on the topic is performed but the idea starts to die down towards the beginning of the 2000s.

Quantum Cellular Automata blend the ideas of quantum mechanics like superposition and entanglement with classical CA systems. Unlike CCA, QCA don't have a concrete definition and, thus, have no clear origin, but rather a series of independent origins that gave their own definition to this concept.

John Von Neumann provided much of quantum mechanic's mathematical formalization, including functional analysis and density matrices. Nevertheless, it is not him that connected the mathematical framework of quantum science and CCA, but rather physicist Richard Feynman. In 1982, Feynman comes up with the concept of a quantum computer as a way to computationally model the

quantum mechanical behaviour of nature. This paved the way for the first rigorous definition of QCA in 1988 by physicists Gerhard Grössing and Anton Zeilinger. They define a quantum Cellular Automata system as a CA system where each cell's state is not governed by a discrete value 0 or 1 but rather a probability amplitude like that of a quantum state. The system evolves according to quantum mechanical rules that permit superposition of these amplitudes across cells. Additionally, the evolution is linear and conserves total probability, but does not require unitary evolution.

3.2 Concepts from physics and mathematics

3.2.1 Fourier Analysis

Fourier analysis is the branch of mathematics that breaks down complex wave functions into simple sine and cosine forms. The power spectrum shows how much power the complex wave has at every frequency. The mathematical formulas for the Fourier transformation are the following, and encapsulate how a complex function is decomposed into the finite or infinite sum of simpler trigonometric functions:

$$X_k = \sum_{n=0}^{N-1} x_n \exp\left(-\frac{i2\pi}{N} kn\right) \quad (1)$$

Figure 1: Formula for the Discrete Fourier Transform (finite form).

$$F(\omega) = \int_{-\infty}^{\infty} f(t) \exp(-i\omega t) dt \quad (2)$$

Figure 2: Formula for the Continuous Fourier Transform (infinite form).

The formula for the power spectrum is the following:

These concepts will be used for classifying the rule numbers.

3.2.2 Classical Information Theory

Classical information theory is the mathematical framework formalized by Claude Shannon in 1948. Within the field it is important to be familiar with the concepts of Shannon entropy, mutual information, effective complexity.

$$P_k = \frac{1}{N^2} |X_k|^2 \quad (3)$$

Figure 3: Finite (Discrete) Form

$$S(\omega) = |F(\omega)|^2 \quad (4)$$

Figure 4: Infinite (Continuous) Form

Figure 5: Concise formulas for the finite and infinite power spectrum.

Shannon entropy is the mathematical measure of uncertainty or randomness within a data set. The function of this parameter is $H(X) = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$, where X is a discrete random variable, x_i is one possible state of the random variable and $p(x_i)$ is the associated probability that that mapping takes place.

Mutual information is how much knowing the state of a random variable reveals about the state of another random variable. Mathematically this is defined the following way:

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right) \quad (5)$$

Figure 6: Formula for mutual information for discrete variables.

$$I(X; Y) = \int_X \int_Y p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right) dx dy \quad (6)$$

Figure 7: Formula for mutual information for continuous variables.

However, in CellDy, the following alternative formulas are used, as to take advantage of the already defined entropy functions:

Finally, effective complexity is a measure of a system's information that focuses on structure and regularities and ignores random or incidental noise. This will be explained more in detail and with examples when these parameters are coded into the library. In CellDy, it is computed as the mutual information between a variable in different timesteps. DS

$$\begin{aligned}
I(X; Y) &= H(X) + H(Y) - H(X, Y) \\
I(X; Y) &= H(X) - H(X|Y) \\
I(X; Y) &= H(Y) - H(Y|X)
\end{aligned}$$

Figure 8: Formulas for mutual information expressed via marginal, joint, and conditional entropies.

3.2.3 Dynamical Systems

Dynamical systems is the branch of mathematics that explores how a system evolves through time under the constraints of certain rules and limitations, just like Cellular Automata.

From the theory of dynamical systems it is important to be familiar with the concepts of damage and the Lyapunov exponent, and Langton's lambda.

Damage is the parameter that measures the difference in two same systems, but where the initial conditions have been slightly changed. The Lyapunov exponent is computed with this formula:

$$\lambda = \frac{1}{t} \ln \frac{|\delta(t)|}{|\delta(0)|}$$

Figure 9: Formula for the Lyapunov Exponent.

The Lyapunov exponent is a parameter that measures the evolution of a system with the following criteria:

- $\lambda < 0$: The system is increasingly more organized
- $\lambda \approx 0$: The system is relatively uniform
- $\lambda > 0$: The system is increasingly more disorganized

Langton's lambda is another parameter, which will be explained later, but it is relevant because it doesn't need to compute the dynamics of the system.

3.2.4 Stochastic Matrices

A Markov chain is a system where there is a given probability of transitioning between states within this system. The key assumption is the Markov property, which states that the evolution remains independent of its previous past, ergo, the current state shares no information regarding the previous evolution of the system:

The evolution for these systems can be modeled via matrix multiplication and the convention is the following:

$$P(X_{t+1} = j \mid X_t = i, X_{t-1}, \dots, X_0) = P(X_{t+1} = j \mid X_t = i) \quad (7)$$

Figure 10: The Markov property, stating that the probability of the next state depends only on the current state.

$$P_{ij} = P(X_{t+1} = j \mid X_t = i) \quad (8)$$

Figure 11: Definition of the transition probability from state i to state j .

3.3 Concepts from Computer Science

Both the variety of different CA rules as well as its potential for simulating problems in physics make certain computer science topics relevant to fully understand the depth of CA and the potential of CellDy.

Firstly, some rules exhibit what is called Turing completeness, meaning they can compute everything that a Turing machine can, ergo, they can act as a computer. Another important definition is universal computation, meaning that a system can compute any possible algorithm. In this specific case, they are actually synonyms.

Rice's theorem states that "any non-trivial semantic property of program is undecidable". In the case of CA it means that any nontrivial question about its long-run behavior is undecidable and there is no shortcut algorithm to fall back on.

This is also related to the concept of computational irreducibility, which applied to CellDy means that there is no way of knowing what happens at timestep "x", except for running the program for "x" timesteps. This is the reason why CellDy is framed as simulator rather than a solver.

3.4 Stephen Wolfram's Classes of Cellular Automata

Like mentioned earlier, Stephen Wolfram's classes are introduced in "Statistical mechanics of cellular automata". These are four classifications Wolfram comes up with in which all different CA rules can be grouped into. These are the following:

- **Class 1** (Converging): The system converges to the same final state regardless of its initial conditions.
- **Class 2** (Periodic or stable): The system evolves into stable or periodic patterns. Small changes in the initial conditions lead to small changes in

the evolution of the system.

- **Class 3** (Chaotic): The system is chaotic and impossible to predict. Even slightly different conditions can drastically reduce the evolution of the state.
- **Class 4** (Complex): The system exhibits complex behaviour with localized structures and complexity. Wolfram describes it as the boundary between order and chaos.

4 Conventions

Beforehand, it is important to make clear the conventions that the library uses in its functions.

Firstly, in the numerical representation of systems, a "0" is a dead cell whilst a "1" is one that is alive. In a plotted system, a black square is a dead cell and a white cell is one that is alive.

Additionally, in a system, the timestep of the system decreases with row number, meaning that the first timesteps are at the bottom and the most recent ones are at the top.

5 Classical Cellular Automata: Notebooks 1, 2 and 3

5.1 Notebook 1: Functions that automate evolution for Cellular Automata

In the first Notebook, the most fundamental functions are created:

• **make_rule(rule_number: int)**: This is the most fundamental function and it describes how the rule number is actually translated into the evolution of a discrete system. The inputted number, in decimal, is translated into binary form using an 8-bit representation. That means there are $2^8 = 256$ different possible rules.

• **complement_rule(rule_number: int)**: Outputs the rule that has the opposite mappings to the rule number that was inputted.

• **dynamics(rule_number, L, initial_conditions, timesteps)**: This function computed the dynamics of a system given the initial conditions, which is the list of positions where there will a one at the beginning timestep, the rule number, the lattice length and the number of timesteps that will be computed.

Boundary cells are an issue with cellular automata that have been treated in different ways, because it is unclear what to do with cells that sit at the edge of the grid. The approach that is taken in **dynamics** is that the empty space in contact with the boundary cells is treated as a zero cell. Nevertheless, **dynamics_periodic** treats empty space as the state of the one neighboring cell they do have and **ddynamics_reflection** treats the empty space as the state of the other boundary cell, both of these are other common ways of dealing with boundary cells.

Visual analysis of the system is done with the function **plot_grid(grid)** that has an output of the dynamics function as its input. It follows the convention that dead cells are black and alive ones are white.

The rest of the first notebook is dedicated to analyzing the variety of rules when additional constraints are imposed on the behaviours. The first constraint is commonly used when modeling physics problems or population growth and it states that three dead cells will always give place to another dead cell regardless of what the rule states. This means that there will be multiple rules that behave the same, as the dynamics will evolve the same way regardless of its mapping for the case of three dead cells. More concretely it cuts the number of unique rules down to 128. This condition is imposed on a rule by **make_rule_zero_condition(rule_number: int)**.

A further constraint can be imposed such that the local evolution of a unique cell is also governed by symmetry such that the mapping of a neighborhood must be the same as that of its cyclic shift. For example "110" and "011" must have the same output. The mappings that are overwritten are the ones more right-leaning. This condition is imposed on a rule by the function **make_rule_symmetry(rule_number: int)**. If both constraints are imposed, then the number of unique rules is reduced to 32.

5.2 Notebook 2: Studying Behaviour in Chaotic Cellular Automata

The second notebook is focused on comparing different systems with each other and finding possible classifications of rule numbers. This is done both visually with functions that allow for the plotting of multiple systems next to each other as well as the computation of a system at only certain timesteps.

It is also done mathematically using principles from information theory, Wolfram's definitions of four distinct classes, dynamical systems and Fourier analysis.

5.2.1 Visual comparison

The function **plot_grid_multiple**(grids: list, cols: int) plots multiple plots within the same grid allowing them to be visualized side by side, the input grids is a list of **dynamics** outputs and cols is the number of columns that the grid will have.

The function **dynamics_initialandfinal**(rule_number, L, initial_conditions, timesteps) outputs the initial conditions of a system, and the system at its last timestep, specially useful for chaotic systems. Furthermore, **state_comparison**(system_1, system_2) outputs the number of cells that is distinct between two systems as well as the quotient between that value and the total number of cells in the system. **state_comparison_slice**(system_1, system_2, timestep, axis) generalizes that to an individual timestep and can be used for rows (timesteps) and columns (individual cells through time).

The function **plot_cell**(rule, cell_i, timesteps, L) visually plots the state of an individual cell through time. The function **plot_column**(arr, col, cols=100) represents it using the same convention as the previous system plots. The function **plot_multiple_cells**(*cells) generalizes the first function to accomodate multiple cells.

5.2.2 Classification using physics

Different fields of physics and mathematics can be used to make comparisons and classifications of CA systems, most of these are then compared to Stephen Wolfram's classes.

Using Fourier Analysis:

The behaviour of rules can be obtained by using principles of Fourier analysis. The function **fourier_cell**(rule_number, L, initial_conditions, timesteps, cell_index, apply_window=True, zero_pad=True) provides the plot of its state through time, the windowed time series and the power spectrum plotted (power vs frequency). **fourier_all_cells**(rule_number, L, initial_conditions, timesteps, apply_window=True, zero_pad=True) calls the same function for all the cells in a system.

The classification scheme is the following:

- **Class 1: The power spectrum is flat.** The system always converges to the same state regardless of the initial conditions, so the cell's power approaches zero and maintains that value.
- **Class 2: The power spectrum exhibits one single peak.** Because the system exhibits predictable, non-trivial behaviour, then the power peaks at a single value corresponding to a certain frequency.

- **Class 3: The power spectrum shows periodic peaks.** Because the system approaches random behaviour, there is no frequency at which the power peaks.
- **Class 4:**

It is important to mention that the code implements Hann windowing and zero padding to guarantee coherent results between rules.

Using Classical Information Theory:

The principles of classical information theory may also be used to classify CA rules, particularly with effective complexity, as it takes a detailed look at how the system evolves and how knowing the initial states can prove useful for later states. The classification criteria is the following:

- **Class 1: Effective complexity is zero.** Because the system always converges, then knowing the initial conditions gives no information on subsequent timesteps.
- **Class 2: Effective complexity is low.** Although the information can be traced back, it is non-complex, but not trivial.
- **Class 3: Effective complexity is zero.** Because the system converges to randomness.
- **Class 4: Effective complexity is high.** The information can be traced back and it is complex.

The formula to calculate the Shannon entropy of a system at every timestep is **H_CA(rule)** and to calculate it for a single cell across the timestep it is **H_cell(rule, location)** where location is the position index of the column where the cell is situated.

H_two_cells(rule, cell_i, cell_j) calculates the joint Shannon entropy. This parameter is used for the computation of mutual information using the function **mutual_information(rule, cell_i, cell_j, L=1000)**
needs to be finished

Using Dynamical Systems:

The damage associated to a rule is given by the function **damage(rule)** and the way it works is it computes a system with said rule and changes the initial conditions minimally, it then computed the damage after a preset number of timesteps. The outputs are the damage at each timestep, the mean and the variance. **plot_damage(rule)** plots the damage at each timestep.

lyapunov_exponent(rule) calculates the Lyapunov exponent, using the damage function.

Like was mentioned in section 3, Lyapunov's exponent points to the evolution of the system and the classification criteria is the following:

- **Class 1: Effective complexity is zero.** Because the system always converges, then knowing the initial conditions gives no information on subsequent timesteps.
- **Class 2: Effective complexity is low.** Although the information can be traced back, it is non-complex, but not trivial.
- **Class 3: Effective complexity is zero.** Because the system converges to randomness.
- **Class 4: Effective complexity is high.** The information can be traced back and it is complex.

Additionally, there is another way of classifying CCA systems that does not have to do with any previous concepts and does not require computing the dynamics, however, it is often unreliable and not mathematically rigorous. Nevertheless, it should be mentioned. This is Langton's Lambda and it is the ratio of neighborhoods that map to one to the total number of neighborhoods, which is eight. This parameter is introduced in the literature "Studying artificial life with cellular automata.", by Cristopher G. Langton. In this same literature he provides the classification for the rules using Langton's lambda:

- $\lambda = 0.0$ Rule 0, class I
- $\lambda = 0.0 - 0.2$ Class I
- $\lambda = 0.2 - 0.4$ Class II
- $\lambda = 0.5$ Class IV, at the edge of chaos
- $\lambda = 0.6 - 0.8$ Class III
- $\lambda = 0.8 - 1.0$ Class I
- $\lambda = 1.0$ Class I, all ones

Langton's lambda of a rule is given by **langtons_lambda(rule)**.

5.2.3 Studying Reversibility for Different Rules

There are reversible and irreversible rules. A rule is reversible if every possible state of the system can only be caused by one single other state (it is a bijection), any rule that doesn't meet this criteria is considered irreversible.

Almost none of the rules are reversible except for arbitrary rules such as rule 170 and 240, which correspond to a left and right shift, respectively, and rule 204, that corresponds to an identity rule. Additionally, depending on lattice length, other rules can coincidentally be reversible. This can be put to test with the rule `is_reversible(rule, L=10)`.

Additionally, a Garden of Eden state is a state to which a rule can never map to, therefore, if it appears in a system governed by said rule, it must appear as the initial conditions. `gardens_of_eden(rule, L=10)` outputs the Garden of Eden states depending on a rule and its lattice length and the number of said states.

5.3 Notebook 3: Building Up to Quantum Cellular Automata - Semiclassical Cellular Automata

`next_state(state, rule_number, L)` outputs the next timestep taking only the current step, rule and lattice length.

A part of Notebook 3, though being mostly about SCA, dedicated the beginning to introducing matrix formalism for CCA and discussing its main constraint: Non-linear time scaling.

This subsection assumes background knowledge of Markov chains and Markov transition matrices. CA are systems that can be modeled via stochastic matrices. The way this is done is that the initial conditions are converted into a probability and the matrix is multiplied with this vector. The new probability vector is then converted back into a state. `vectorized_ic(L, initial_conditions)` does this conversion into a probability vector and the matrix is computed with `markov_matrix(rule_number: int, L: int)`. These functions are incorporated into `dynamics_matrix(rule_number, L, initial_conditions, timesteps)`, which yields the same output as the original `dynamics` function, but uses matrix multiplication to formalize it.

`rule_matrix_heatmap(rules, L, titles=None)` gives the heatmap of the matrix of the associated rule.

Matrix multiplication for modeling the dynamics is not suggested for two main reasons:

- **Matrix multiplication scales non-linearly:** The computational complexity of matrix multiplication is $O(n^3)$, so the wait time gets too long for larger matrices.
- **Building the matrix is infeasible for large-lattice systems:** The matrix dimensions is $2^L \times 2^L$, so as the lattice length increases the matrix becomes too big for convenient computation.

6 Semiclassical Cellular Automata

6.1 Notebook 3

Semiclassical Cellular Automata are created as a concept that is meant to approach the idea of quantum Cellular Automata without abandoning the original essence of the classical dynamics. The definition that is finally given to SCA is that they are systems analogous to their fully classical counterparts, but there is an inputted probability applied to each cell in the current timestep that its state will flip. The computation of the subsequent timestep is done with the current timestep after these probabilities are applied to it. The reason this is done is because what was wanted was a system that would be affected by a certain inputted randomness and the dynamics would evolve taking this randomness into account, hence why it is applied before the neighborhoods are mapped onto the next timestep.

`semiclassical_markov_matrix(rule_number: int, L: int, probability: float)` computes the matrix associated to this probabilistic dynamics, where `probability` is the probability applied to each state that it is flipped, and works with the original `vectorized_ic(L, initial_conditions)` function. However, `semiclassical_dynamics(rule_number, L, initial_conditions, timesteps, probability)` yields the same output without its associated constraint.

7 Quantum Cellular Automata

Bibliography

Section 2: Relevant Background

Classical and Quantum Cellular Automata

- Sarkar, P. (2000). *A brief history of cellular automata*. ACM Computing Surveys, 32(1), 80–107.
- Arrighi, P. (2019). *An overview of Quantum Cellular Automata*. arXiv preprint arXiv:1904.12956.
- Ilachinski, A. (2001). *Cellular Automata: A Discrete Universe*. World Scientific.

Concepts from physics and mathematics

- Stein, E. M., & Shakarchi, R. (2003). *Fourier Analysis: An Introduction*. Princeton University Press.
- Strogatz, S. H. (2015). *Nonlinear Dynamics and Chaos*. Westview Press.
- Wolfram, S. (1983). *Statistical Mechanics of Cellular Automata*. Reviews of Modern Physics, 55(3), 601–644.

Concepts from computer science

- Cover, T. M., & Thomas, J. A. (2006). *Elements of Information Theory*. Wiley-Interscience.

Stephen Wolfram’s Classes of Cellular Automata

- Wolfram, S. (1983). *Statistical Mechanics of Cellular Automata*. Reviews of Modern Physics, 55(3), 601–644.
- Langton, C. G. (1990). *Computation at the Edge of Chaos: Phase Transitions and Emergent Computation*. Physica D: Nonlinear Phenomena, 42(1–3), 12–37.